Университет ИТМО

Домашнее задание № 2

ОТЧЁТ

о воспроизведении результатов статьи

«Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates»

L. N. Smith, N. Topin, arXiv:1708.07120

Выполнил: Александров Андрей Алексеевич

Санкт-Петербург, 2026

1. Введение

Статья Л. Смита и Н. Топина [1] описывает режим обучения, при котором сеть за один цикл learning rate с очень большим максимальным значением обучается примерно на порядок быстрее, чем при обычном ступенчатом расписании, и при этом не теряет в точности. Авторы назвали это явление super-convergence. Для ResNet-56 на CIFAR-10 они приводят 92,4 % точности на тесте за 10 000 итераций против 91,2 % за 80 000 итераций при стандартном обучении.

Цель работы – воспроизвести эти результаты и проверить, насколько они зависят от постановки эксперимента, в первую очередь от того, с каким бейзлайном сравнивается новый режим. Почти все эксперименты статьи с ResNet-56 на CIFAR-10 повторены в PyTorch на одной видеокарте: основное сравнение (рис. 1a статьи), влияние длины цикла, LR range test, влияние weight decay, максимального learning rate и объёма обучающей выборки. Кроме того, поставлен контроль, которого в статье нет: обычное расписание с тем же числом итераций, что и у super-convergence.

2. Краткое содержание статьи

2.1 Циклический learning rate и LR range test

Работа опирается на два инструмента, предложенных ранее первым автором [3]. LR range test – это короткий пробный запуск, в котором learning rate линейно растёт от нуля до большого значения, а по ходу измеряется точность. Обычно точность сначала растёт, затем падает, и значение в точке максимума берут как верхнюю границу для циклического режима. Для ResNet-56 на CIFAR-10 картина необычная: точность остаётся высокой вплоть до learning rate, равного 3, что на порядок больше привычных 0,1. Авторы считают такое поведение признаком того, что сеть можно обучать с очень большим шагом.

В циклическом режиме (CLR) learning rate линейно меняется между нижней и верхней границей: половину цикла (stepsize) растёт, половину убывает. Для super-convergence используется один цикл, который авторы называют 1cycle: learning rate поднимается от 0,1 до 3 и возвращается обратно, а если после цикла остаются итерации, опускается ещё на несколько порядков. В основных экспериментах с ResNet-56 цикл занимает всё обучение – 10 000 итераций при stepsize 5000. Momentum в этих опытах постоянный и равен 0,9; в экспериментах из таблицы 2 статьи он меняется в противофазе к learning rate, от 0,95 до 0,85.

2.2 Большой learning rate как регуляризация

Второй тезис статьи – большой learning rate регуляризует обучение. В LR range test на участке от 0,2 до 2 потери на обучающей выборке растут, а на тестовой падают (рис. 8a статьи). Отсюда авторы делают практический вывод: суммарная регуляризация должна быть сбалансирована, и если её добавляет большой шаг, то остальные виды – weight decay, dropout, малый батч – нужно ослабить. В подтверждение приводится опыт с weight decay: при 10^{-3} super-convergence не возникает, при 10^{-4}…10^{-6} возникает (рис. 12b статьи). На

ImageNet для обучения с learning rate до 1,0 weight decay пришлось уменьшить до $3 \cdot 10^{-6}$… 10^{-5}.

2.3 Оценка оптимального learning rate

В разделе 4 статьи предложена оценка learning rate по кривизне функции потерь. Оптимальный шаг вдоль направления спуска обратно пропорционален кривизне в этом направлении, а кривизну можно оценить по разности градиентов в двух точках, как в методе AdaSecant [7]. Если выразить градиенты через веса с помощью правила обновления $\theta_{i+1} = \theta_i - \varepsilon g_i$, где g_i – градиент функции потерь, получается оценка, которая зависит только от весов на трёх соседних итерациях:

$$\varepsilon^* = \varepsilon \cdot \Sigma|\theta_{i+1} - \theta_i| \,/\, \Sigma|2\theta_{i+1} - \theta_i - \theta_{i+2}|, \qquad (1)$$

где ε – learning rate, с которым шло обучение, а суммы берутся по всем весам сети. Числитель – величина шага, знаменатель – его изменение между итерациями. Если шаг почти не меняется, поверхность потерь плоская и оценка получается большой. Для ResNet-56 авторы получили значения от 2 до 6 и интерпретировали это как признак того, что большой learning rate приводит в широкие плоские минимумы. Полноценно этот метод в статье не проверялся.

2.4 Проверяемые результаты

В таблице 1 собраны результаты статьи для ResNet-56 на CIFAR-10, которые проверяются в этой работе.

Таблица 1 – Результаты статьи для ResNet-56 на CIFAR-10

Эксперимент	Обычное обучение	Super-convergence	Источник
Основное сравнение	91,2 % (80 000 ит.)	92,4 % (10 000 ит.)	рис. 1a
Длина цикла 2000–20 000 итераций	–	89,7–92,7 %	табл. 6, рис. 1b
Верхняя граница LR от 1 до 3,5	–	90,9–92,4 %, лучшая при 3	табл. 6
Weight decay 10^{-3}	–	эффекта нет	рис. 12b
10 000 обучающих примеров	71,4 %	80,6 %	табл. 1
20 000 / 30 000 / 40 000 примеров	82,7 / 85,7 / 89,1 %	87,9 / 89,6 / 91,1 %	табл. 1

3. Постановка эксперимента

3.1 Реализация

Авторы использовали Caffe и кластер с восемью GPU на узел; батч 1000 делился между картами. Все эксперименты этой работы выполнены в PyTorch 2.14 (CUDA 13.0) на одной видеокарте NVIDIA GeForce RTX 5060 Ti с 16 ГБ памяти. Код разбит на несколько модулей: загрузка данных, модели, расписания learning rate, оценка по формуле (1), цикл обучения и построение графиков. Каждый запуск сохраняет свои параметры, кривые обучения с шагом 200 итераций и итоговые метрики, так что все числа в отчёте можно пересчитать по логам (приложение А).

CIFAR-10 целиком помещается в память видеокарты, поэтому данные загружаются на GPU один раз, а случайный сдвиг (дополнение на 4 пикселя и вырезка 32×32) и

горизонтальное отражение делаются прямо на GPU. Без этого при батче 1000 узким местом становится загрузка данных. Обучение идёт в смешанной точности bfloat16. Одна итерация ResNet-56 с батчем 1000 занимает около 95 мс: 10 000 итераций – около 17 минут, 80 000 – около двух часов. Поэтому эксперименты удалось провести в том же масштабе, что и в статье.

3.2 Модель и данные

Используется стандартная для CIFAR архитектура ResNet-56 [2]: три стадии по девять остаточных блоков с 16, 32 и 64 каналами, 0,85 млн параметров.

Обучающая выборка CIFAR-10 [6] содержит 50 000 изображений, тестовая – 10 000. Для опытов с ограниченными данными берутся случайные подвыборки по 10 000, 20 000, 30 000 и 40 000 изображений с одинаковым числом примеров каждого класса; подвыборка фиксирована и одинакова для всех режимов. Точность везде измеряется на полном тестовом наборе.

3.3 Режимы обучения

Сравниваются три режима (таблица 2, рисунок 1). Во всех используется SGD с momentum 0,9 и weight decay 10^{-4}, если не указано иное.

Таблица 2 – Режимы обучения

Режим	**Расписание learning rate**	**Батч**	**Итераций**
CLR (super-convergence)	линейно 0,1 → 3 → 0,1, один цикл на всё обучение	1000	2000–20 000
Ступенчатый (PC)	0,35 с линейным разогревом 200 итераций, ×0,1 на 50 % и 75 % обучения	1000	10 000, 20 000, 80 000
Бейзлайн He et al. [2]	0,1, ×0,1 на 32 000 и 48 000 итерациях	128	64 000

Ступенчатый режим с learning rate 0,35 и 80 000 итерациями – это бейзлайн статьи. Моменты снижения learning rate в статье не указаны, сказано лишь, что его уменьшают в 10 раз, когда точность выходит на плато; здесь выбраны 50 % и 75 % обучения. Разогрев понадобился потому, что без него сеть не обучается вовсе (раздел 4.8). Третий режим – стандартное расписание из статьи о ResNet [2], для которого опубликована точность 93,03 %. Он нужен, чтобы понять, насколько силён бейзлайн статьи.

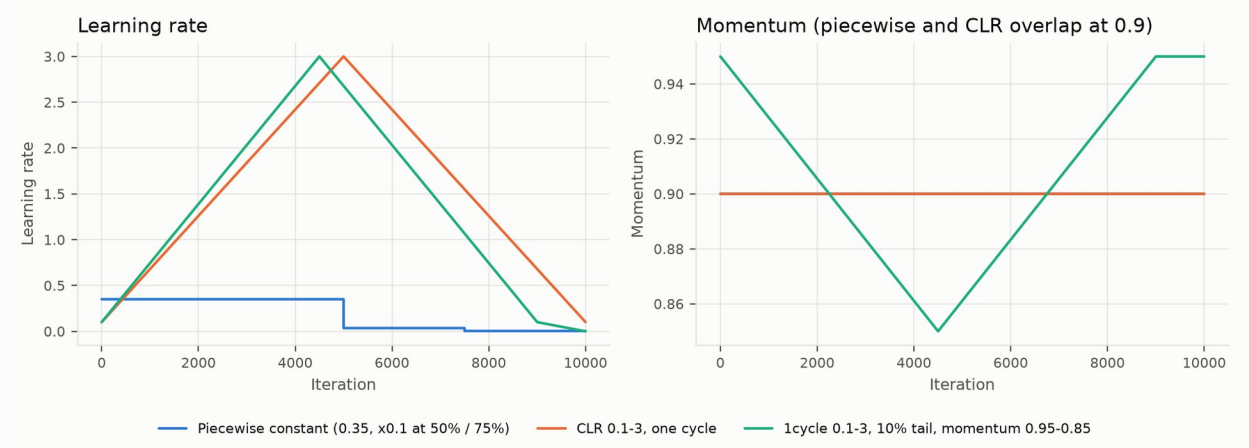


Рисунок 1 – Расписания learning rate и momentum на 10 000 итераций: ступенчатое (PC), один цикл CLR и 1cycle с завершающим участком и циклическим momentum (последний вариант в экспериментах не использовался)

3.4 Отличия от оригинальной постановки

Таблица 3 – Отличия от постановки статьи

Аспект	**Статья**	**Эта работа**
Фреймворк и оборудование	Caffe, 8 GPU, батч делится на 8 частей	PyTorch, 1 GPU, статистики BatchNorm по всему батчу
Обновление SGD с momentum	$v \leftarrow \mu v + \varepsilon g,\ \theta \leftarrow \theta - v$	$v \leftarrow \mu v + g,\ \theta \leftarrow \theta - \varepsilon v$
Сглаживание статистик BatchNorm	0,95 у CLR, 0,999 у бейзлайна	0,95 во всех режимах
Разогрев бейзлайна	не упоминается	200 итераций
Короткий путь при понижении разрешения	avgpool 3×3 и дополнение нулями	прореживание и дополнение нулями (вариант A [2])
Точность вычислений	float32	bfloat16

При постоянном learning rate обе формы обновления SGD эквивалентны. При циклическом расписании PyTorch применяет новый learning rate ко всей накопленной скорости, но за одну итерацию learning rate меняется лишь на 0,0006, так что разница пренебрежимо мала. Влияние bfloat16 проверялось отдельно на случае, где оно могло бы сказаться сильнее всего (раздел 4.8).

4. Результаты

Во всех таблицах приводится точность на тестовой выборке в конце обучения, а не максимальная по ходу: выбор лучшей точки по тестовым данным завышал бы результат. Чтобы оценить разброс, основные конфигурации запущены с тремя сидами. Стандартное отклонение для CLR на 10 000 итераций составило 0,46 п. п., поэтому различия меньше примерно 1 п. п. между одиночными запусками нельзя считать значимыми.

4.1 Основное сравнение

Рисунок 2 воспроизводит рисунок 1a статьи. CLR за 10 000 итераций дал 92,95 % (в среднем по трём сидам 92,52 ± 0,46 %), что совпадает с 92,4 % в статье. Кривая похожа на авторскую: пока learning rate близок к максимуму, точность на тесте сильно колеблется, от 53 до 83 %, и быстро растёт на последних тысячах итераций, когда шаг уменьшается.

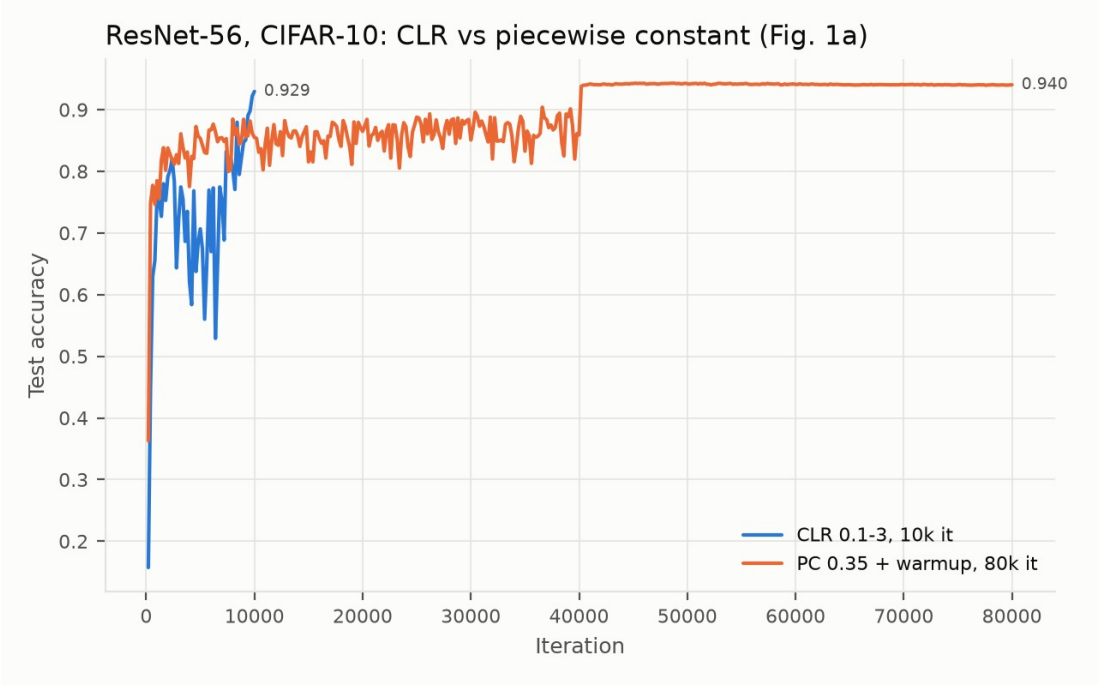


Рисунок 2 – Точность на тесте для CLR 0,1–3 (10 000 итераций) и ступенчатого режима (80 000 итераций), аналог рис. 1a статьи

Бейзлайн же получился намного сильнее авторского: 94,02 % вместо 91,2 %. Пока learning rate равен 0,35, точность не поднимается выше 90,4 % и сильно шумит. Через 200 итераций после первого снижения learning rate она скачком растёт с 85,9 до 93,8 %, а к 41 000-й итерации достигает 94,2 %. Последняя фаза с learning rate 0,0035 ничего не добавляет: средняя точность на итерациях 60 000–80 000 равна 94,04 % против 94,18 % на предыдущем участке, а потери на тесте медленно растут от минимума 0,249 до 0,277. К этому моменту сеть полностью запоминает обучающую выборку (потери около $4 \cdot 10^{-4}$), но точность на тесте почти не меняется.

Таким образом, точность CLR воспроизводится, а вывод о том, что super-convergence точнее обычного обучения, – нет: в нашей постановке обычное обучение на 80 000 итерациях лучше на 1,5 пункта.

4.2 Сравнение при равном числе итераций

Скачок точности сразу после снижения learning rate говорит о том, что бейзлайну мешает не медленная сходимость, а шум большого шага, и время до хорошей точности определяется тем, когда шаг уменьшают. Чтобы это проверить, ступенчатый режим был запущен с тем же бюджетом, что и CLR, – 10 000 и 20 000 итераций с теми же относительными моментами снижения (таблица 4, рисунок 3).

Таблица 4 – Сравнение режимов при разном бюджете

Режим	**Итераций × батч**	**Эпох**	**Точность, %**	**Время**
CLR 0,1–3	10 000 × 1000	200	92,52 ± 0,46 (92,95; 92,56; 92,04)	17 мин
Ступенчатый	10 000 × 1000	200	92,26 ± 0,38 (92,43; 92,52; 91,82)	16 мин
CLR 0,1–3	20 000 × 1000	400	93,12	31 мин
Ступенчатый	20 000 × 1000	400	93,24	31 мин

Режим	Итераций × батч	Эпох	Точность, %	Время
Ступенчатый (протокол статьи)	80 000 × 1000	1600	94,02	2 ч 1 мин
Бейзлайн He et al.	64 000 × 128	164	92,62 (в [2]: 93,03)	24 мин

При равном бюджете CLR и ступенчатый режим практически не различаются. На 10 000 итерациях разница средних составляет 0,26 п. п. при стандартном отклонении около 0,4 (t-критерий Уэлча ≈ 0,76), на 20 000 итерациях ступенчатый режим даже немного впереди.

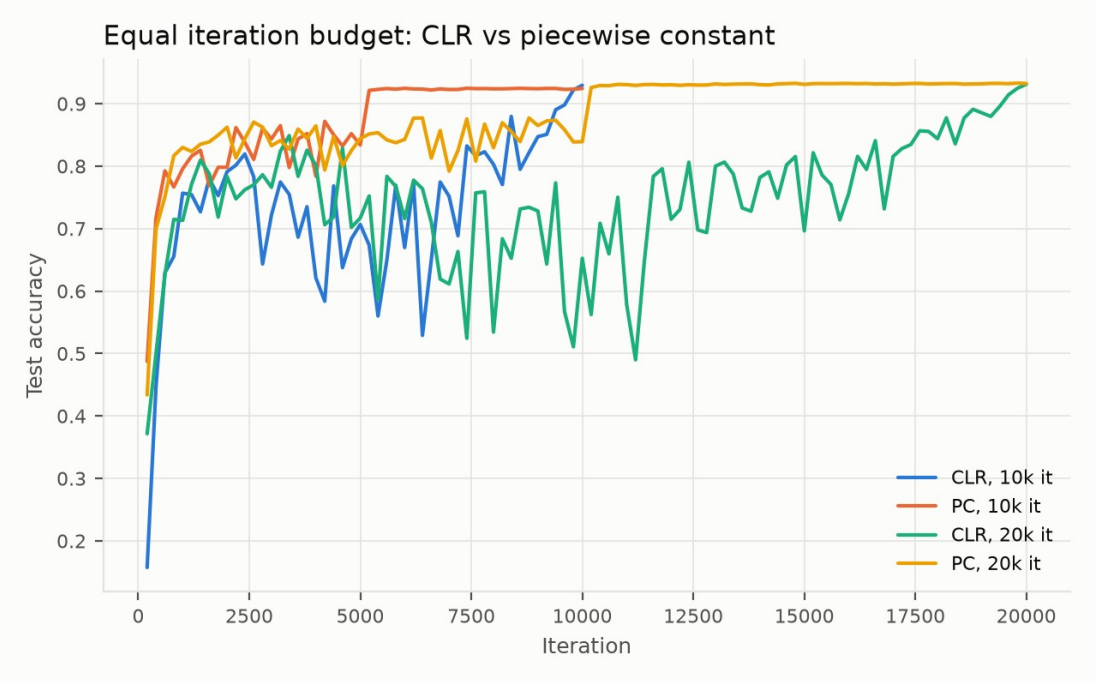


Рисунок 3 – Точность на тесте для CLR и ступенчатого режима при одинаковом числе итераций

Полезно пересчитать бюджеты в эпохи. 80 000 итераций с батчем 1000 – это 1600 эпох, тогда как стандартное расписание из [2] обучает ResNet-56 за 164 эпохи. Наш запуск этого расписания дал 92,62 %, то есть ту же точность, что CLR, при меньшем числе эпох (164 против 200). CLR выигрывает у него только по времени, 17 минут против 24, но этот выигрыш дают большой батч и более полная загрузка видеокарты: ступенчатый режим с тем же батчем идёт так же быстро. К тому же learning rate 0,35 при батче 1000 близок к правилу линейного масштабирования из [4] ($0{,}1 \cdot 1000/256 \approx 0{,}39$), так что ступенчатый режим с разогревом – это обычный рецепт обучения с большим батчем.

Получается, что ускорение «в восемь раз» в статье – результат сравнения с очень длинным и, судя по 91,2 %, плохо настроенным бейзлайном. Обучить ResNet-56 до 92,5 % за 200 эпох действительно можно, но для этого не обязателен именно циклический режим.

4.3 Длина цикла

Таблица 5 и рисунок 4 повторяют опыт с разной длиной цикла (таблица 6 и рис. 1b статьи). Точность растёт с длиной обучения, от 90,93 % на 2000 итерациях до 93,12 % на 20 000, и на всех длинах не ниже, чем у авторов. В статье вместе с длиной цикла менялся

коэффициент сглаживания статистик BatchNorm (от 0,80 для 2000 итераций до 0,97 для 20 000); здесь он везде равен 0,95, чтобы менялся только один параметр.

Таблица 5 – Точность CLR 0,1–3 (%) в зависимости от длины обучения

Итераций	**2000**	**4000**	**6000**	**8000**	**10 000**	**20 000**
Эта работа	90,93	92,30	92,13	92,65	92,52 ± 0,46	93,12
Статья	89,7	91,1	92,1	91,7	92,4	92,7

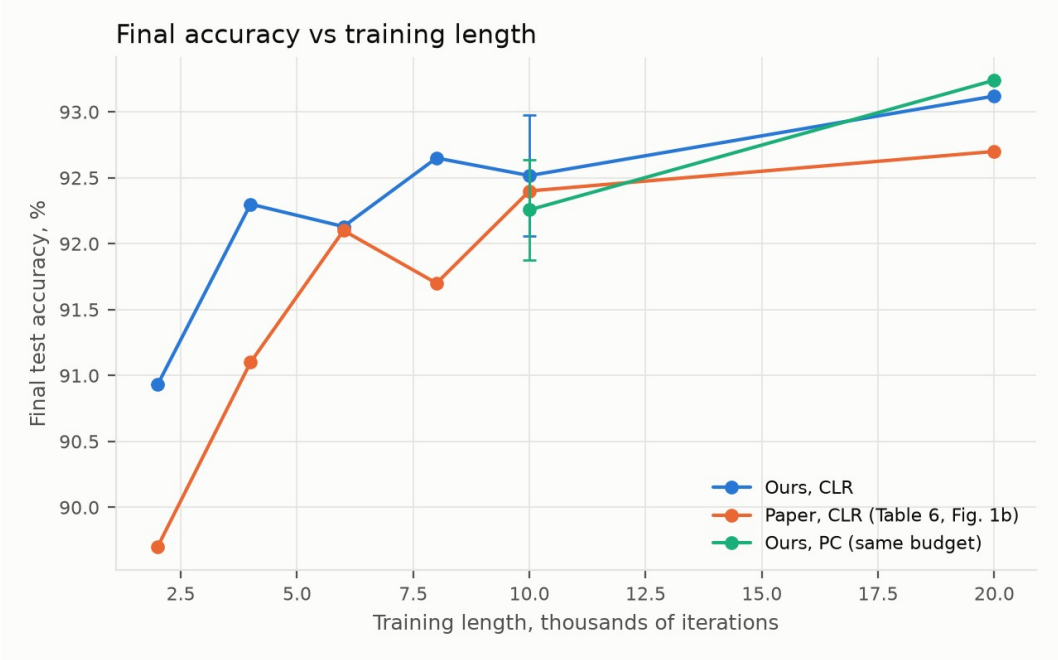


Рисунок 4 – Итоговая точность в зависимости от длины обучения. Для 10 000 итераций показаны среднее и стандартное отклонение по трём сидам

4.4 LR range test

LR range test проведён дважды: learning rate линейно увеличивался от 0 до 3 за 5000 и за 20 000 итераций (рисунок 5, таблица 6). Главное наблюдение статьи воспроизводится: точность на тесте не имеет выраженного пика и держится на уровне 70–85 % во всём диапазоне до 3. Независимость от длины теста, которую отмечают авторы, подтвердилась хуже: при 20 000 итерациях точность в среднем на 4–11 п. п. выше, потому что сеть дольше обучается, хотя форма кривых похожа.

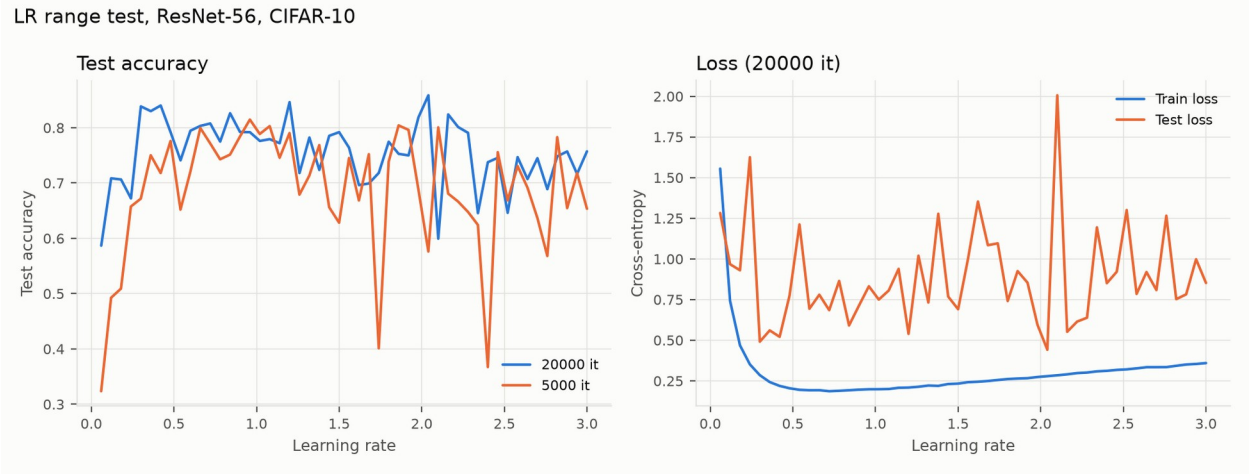


Рисунок 5 – LR range test: точность на тесте для двух длин теста (слева) и потери на обучении и тесте для теста на 20 000 итераций (справа)

Наблюдение о регуляризации воспроизводится только наполовину. Потери на обучении минимальны около learning rate 0,7 и дальше растут, то есть большой шаг не даёт сети подстроиться под обучающие данные. Но потери на тесте при этом не уменьшаются, как на рис. 8a статьи, а слегка растут и сильно шумят, с отдельными выбросами до 2,0. Разность потерь на тесте и на обучении остаётся примерно постоянной, 0,53–0,70, так что уменьшения ошибки обобщения с ростом learning rate здесь не видно.

Таблица 6 – Средние значения по диапазонам learning rate в LR range test на 20 000 итераций

Learning rate	**0,2–0,5**	**0,5–1**	**1–1,5**	**1,5–2**	**2–2,5**	**2,5–3**
Точность, % (средн. / макс.)	79,5 / 84,0	79,2 / 82,6	77,5 / 84,6	74,7 / 81,9	75,0 / 85,9	72,4 / 75,7
Потери на обучении	0,261	0,193	0,215	0,257	0,299	0,340
Потери на тесте	0,795	0,796	0,836	0,956	0,902	0,940

4.5 Weight decay

Опыт с weight decay (рис. 12b статьи) воспроизводится хорошо (таблица 7, рисунок 6). При weight decay 10^{-3} обучение разваливается: лучшая точность 65,1 % достигается на 1200-й итерации при learning rate 0,8, дальше она скачет, к пику learning rate потери на обучении доходят до 1,66, после чего сеть перестаёт обучаться совсем – потери становятся равны ln 10, точность падает до 10 %, и даже возвращение learning rate к 0,1 её не восстанавливает. При 10^{-4}, 10^{-5} и 10^{-6} обучение проходит нормально; лучший результат при 10^{-4} и 10^{-5}, при 10^{-6} точность ниже примерно на 0,7 п. п., что на границе разброса.

Таблица 7 – Точность CLR 0,1–3 на 10 000 итерациях при разном weight decay

Weight decay	**10^{-3}**	**10^{-4}**	**10^{-5}**	**10^{-6}**
Точность, %	10,0 (обучение развалилось)	92,52 ± 0,46	92,65	91,78

Интереснее форма кривых. Пока learning rate большой (итерации 2000–8000), точность при weight decay 10^{-5} и 10^{-6} держится на уровне 77–91 % и почти не шумит, а при 10^{-4} колеблется от 53 до 83 %: чем сильнее weight decay, тем шумнее обучение. Это согласуется с известным эффектом для сетей с BatchNorm [5]. Веса перед нормализацией

инвариантны к масштабу, поэтому их эффективный learning rate пропорционален $\varepsilon/\|w\|^2$, а weight decay уменьшает норму весов и тем самым увеличивает эффективный шаг. Learning rate и weight decay в такой сети действуют не независимо, а усиливают друг друга. Этим можно объяснить и развал при 10^{-3}, и «баланс регуляризации», о котором пишут авторы.



Рисунок 6 – Точность на тесте для CLR 0,1–3 (10 000 итераций) при разном weight decay

4.6 Верхняя граница learning rate

Таблица 8 и рисунок 7 соответствуют таблице 6 статьи. Как и у авторов, верхняя граница 1,0 даёт худший результат, а лучшие получаются в районе 2,5–3. Но зависимость здесь заметно более плоская: все значения от 1,5 до 3,5 укладываются в 1,1 п. п., что сравнимо с разбросом между сидами. Утверждение, что для super-convergence нужен learning rate порядка 3, в нашей постановке выглядит слабее: уже при 1,5 точность составляет 93,1 %.

Таблица 8 – Точность CLR (%) на 10 000 итерациях в зависимости от верхней границы learning rate

Верхняя граница	**1,0**	**1,5**	**2,0**	**2,5**	**3,0**	**3,5**
Эта работа	92,02	93,11	93,26	93,59	92,52 ± 0,46	92,95
Статья	91,3	90,9	91,7	92,3	92,4	92,1

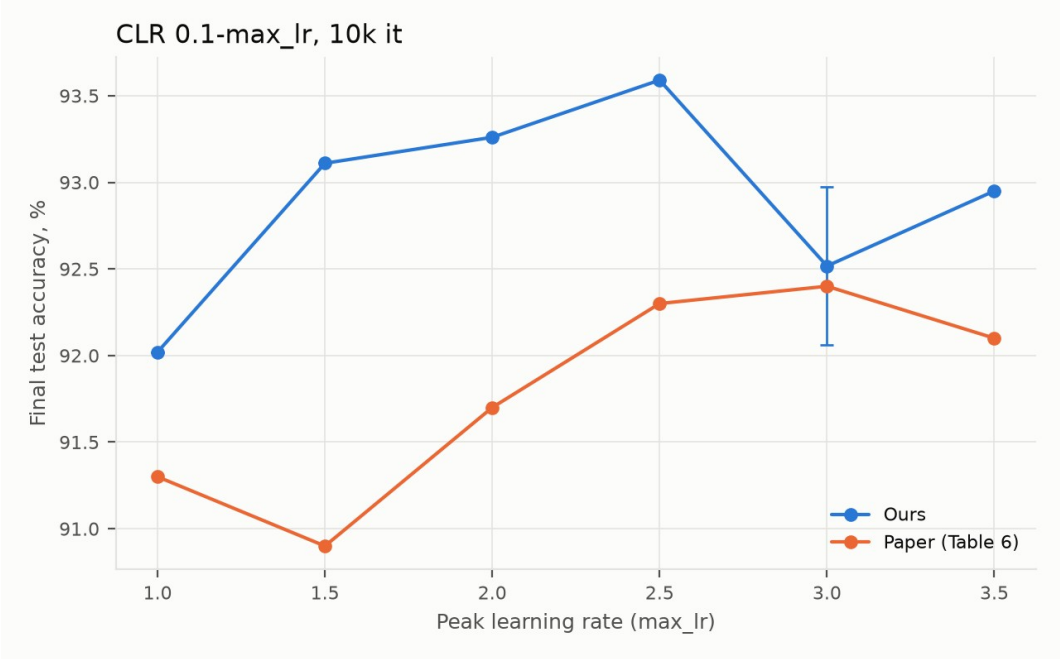


Рисунок 7 – Итоговая точность в зависимости от верхней границы learning rate; для 3,0 – среднее по трём сидам

4.7 Ограниченный объём данных

Самое сильное утверждение статьи касается обучения на части данных: на 10 000 примерах CLR обгоняет обычное обучение на 9,2 п. п. Здесь оно проверено в двух вариантах: ступенчатый режим по протоколу статьи (80 000 итераций) и с тем же бюджетом, что у CLR (10 000 итераций). Результаты приведены в таблице 9 и на рисунке 8; на подвыборках каждый режим запускался один раз.

Таблица 9 – Точность (%) при обучении на части выборки

Обучающих примеров	**10 000**	**20 000**	**30 000**	**40 000**	**50 000**
CLR, 10 000 ит.	86,24	90,18	91,93	92,67	92,52
Ступенчатый, 10 000 ит.	83,76	88,11	90,23	91,71	92,26
Разница CLR − ступенчатый	+2,48	+2,07	+1,70	+0,96	+0,26
Ступенчатый, 80 000 ит.	87,38	90,87	92,46	93,02	94,02
Статья: CLR, 10 000 ит.	80,6	87,9	89,6	91,1	92,4
Статья: ступенчатый, 80 000 ит.	71,4	82,7	85,7	89,1	91,2

При равном бюджете тренд статьи воспроизводится: преимущество CLR монотонно растёт с уменьшением выборки, от 0,26 п. п. на полных данных до 2,48 п. п. на 10 000 примерах. Это согласуется с идеей, что регуляризация большим шагом важнее, когда данных мало. Против протокола статьи результат обратный: ступенчатый режим на 80 000 итерациях на всех подвыборках точнее CLR, на 0,35–1,14 п. п. В статье этот режим на 10 000 примерах «сталкивается с трудностями и расходится» и даёт 71,4 %, у нас он обучается стабильно и даёт 87,38 %. Как и на полных данных, точность ступенчатого режима до первого снижения learning rate ограничена шумом (не выше 82,8; 86,2; 88,0 и 89,7 % для четырёх подвыборок) и резко растёт сразу после снижения.

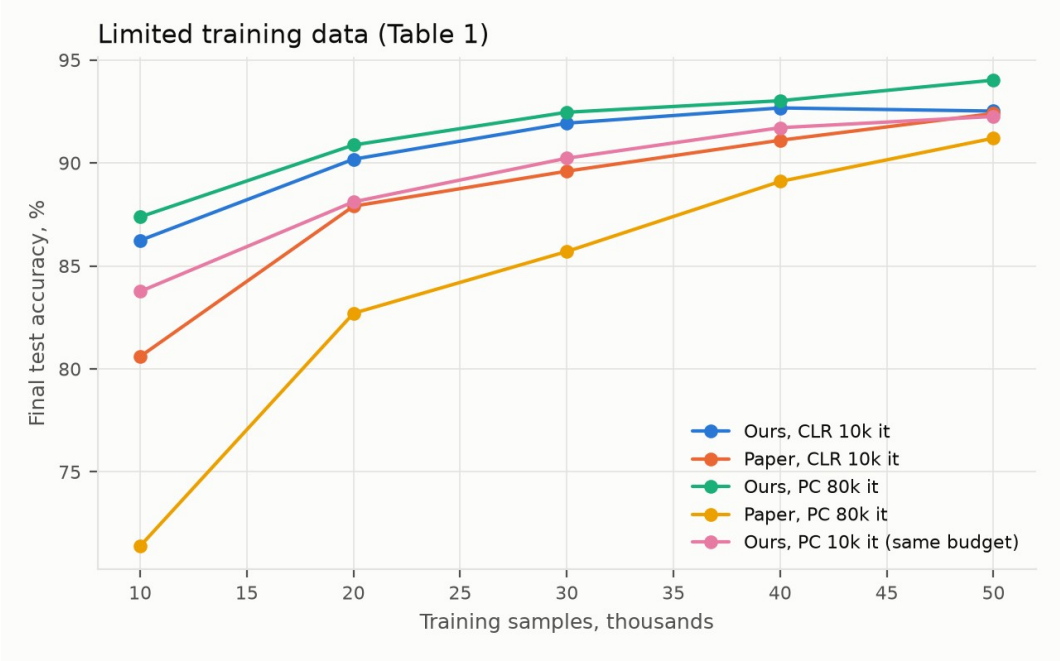


Рисунок 8 – Итоговая точность при обучении на части выборки: CLR и ступенчатый режим в этой работе и в статье

4.8 Дополнительные наблюдения

4.8.1 Разогрев бейзлайна

Если начинать обучение сразу с learning rate 0,35 при батче 1000, сеть за первые 100 итераций приходит в вырожденное состояние: потери на обучении застывают на ln 10 ≈ 2,3026, то есть сеть выдаёт одинаковый ответ для любого входа (таблица 10, рисунок 9). В float32 происходит то же самое, так что дело не в пониженной точности вычислений. С линейным разогревом на 200 итераций тот же режим обучается нормально. В статье бейзлайн с learning rate 0,35 обучался без проблем, значит, в оригинальной постановке было что-то, не описанное в тексте. В CLR роль разогрева играет начальный участок цикла, где learning rate растёт от 0,1; сами авторы называют разогрев дискретным вариантом CLR.

Таблица 10 – Короткие запуски с постоянным learning rate (значения в конце запуска)

Запуск	Итераций	Потери на обучении	Точность на тесте, %
LR 0,35, float32	500	2,3026	10,1
LR 0,35, bfloat16	500	1,892	24,6
LR 0,35, разогрев 200 итераций	600	0,495	77,0
LR 0,1	600	0,985	66,3

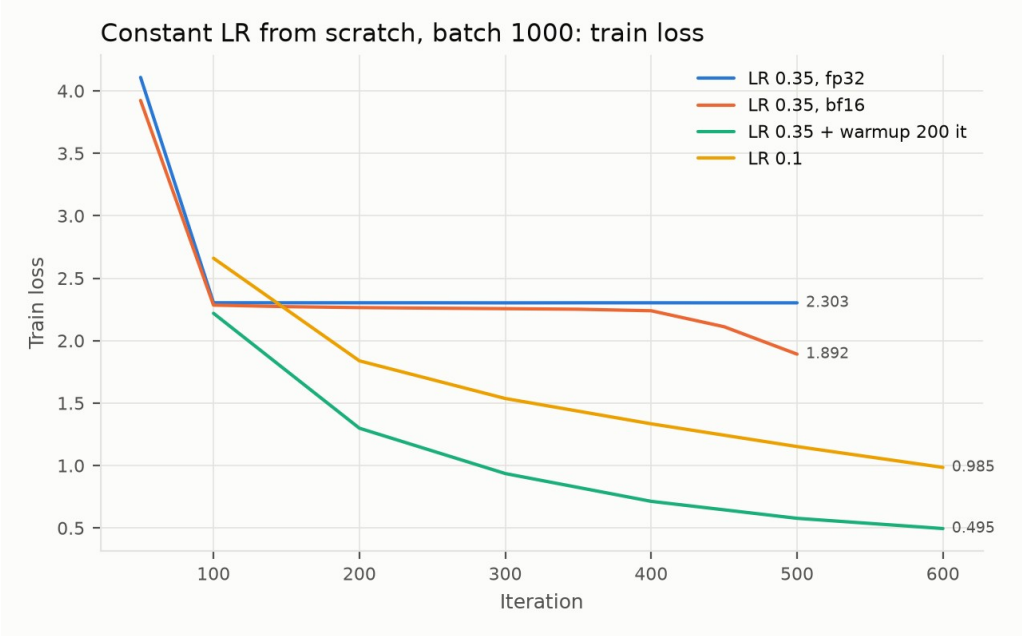


Рисунок 9 – Потери на обучении в первых сотнях итераций при постоянном learning rate

4.8.2 Оценка learning rate по формуле (1)

Формула (1) реализована так же, как в статье: веса трёх соседних итераций, суммирование модулей по всем параметрам, экспоненциальное сглаживание с коэффициентом 0,1. На квадратичной функции $f(w) = \frac{1}{2}h\|w\|^2$ при обычном градиентном спуске она даёт ровно $1/h$, что подтверждает правильность реализации. На ResNet-56 оценка получена только в пробном запуске, где весь цикл CLR 0,1 → 3 → 0,1 сжат в 300 итераций, поэтому напрямую с рис. 3 статьи она не сравнивается. Порядок величины совпал с авторским: медиана сглаженной оценки 7,6 (при сумме квадратов вместо модулей – 4,3), у авторов 2–6.

5. Обсуждение

Сводка результатов приведена в таблице 11.

Таблица 11 – Итоги воспроизведения

Утверждение статьи	**Результат**
CLR 0,1–3 за 10 000 итераций даёт около 92,4 %	подтверждается: 92,52 ± 0,46 %
Super-convergence точнее обычного обучения	не подтверждается: бейзлайн даёт 94,02 %
Обучение на порядок быстрее	только относительно бейзлайна на 1600 эпох; при равном бюджете разницы нет
Точность растёт с длиной цикла	подтверждается
В LR range test точность высока вплоть до LR 3	подтверждается
Большой learning rate уменьшает ошибку обобщения	частично: потери на обучении растут, на тесте не падают
Weight decay 10^{-3} подавляет super-convergence	подтверждается
Оптимальная верхняя граница около 3	частично: лучшая 2,5, но зависимость плоская
На малых данных выигрыш больше	при равном бюджете да (до 2,5 п. п.), против бейзлайна статьи нет

Количественно CLR воспроизвёлся хорошо: почти во всех опытах наша точность не ниже авторской. Расходятся выводы, и почти всегда по одной причине – из-за бейзлайна. В статье обычное обучение даёт 91,2 %, что заметно ниже 93,03 %, опубликованных для той же сети в [2]. Возможные причины видны из таблицы 1 статьи: у бейзлайна статистики BatchNorm сглаживались с коэффициентом 0,999, который сами авторы считают неподходящим для быстрого обучения, а о разогреве ничего не сказано. В нашей реализации тот же режим обучается до 94,02 %. На таком фоне преимущество CLR исчезает, а при равном бюджете остаётся только на малых выборках.

Практическая ценность метода при этом сохраняется. CLR не требует подбирать моменты снижения learning rate, а LR range test за несколько минут показывает, какой диапазон шага допустим. Опыты с weight decay и малыми выборками согласуются с идеей статьи о том, что большой learning rate работает как регуляризатор и его нужно уравновешивать другими видами регуляризации.

При разборе статьи замечены и мелкие неточности: опечатка с шагом первой свёртки в таблице 5, перепутанные ссылки на AdaGrad и AdaDelta (AdaGrad предложен Duchi et al., AdaDelta – Zeiler), запись формулы (6) как деления на вектор. Формула (8) выведена для градиентного спуска без momentum, хотя все эксперименты шли с momentum 0,9, и в статье это не обсуждается.

Ограничения работы. Все опыты выполнены для одной пары ResNet-56 и CIFAR-10. Большинство конфигураций запускалось с одним сидом, с тремя – только сравнение на 10 000 итераций. Реализация отличается от оригинальной в деталях, перечисленных в таблице 3; наиболее существенные из них – разогрев бейзлайна и одинаковый коэффициент сглаживания BatchNorm для всех режимов. Оценка learning rate по кривизне проверена только в пробном запуске.

6. Выводы

1) Основной результат статьи для ResNet-56 на CIFAR-10 воспроизводится: CLR с learning rate от 0,1 до 3 за 10 000 итераций (200 эпох) даёт 92,52 ± 0,46 % против 92,4 % у авторов. Зависимости от длины цикла, верхней границы learning rate и weight decay согласуются со статьёй.
2) Утверждение, что super-convergence точнее и на порядок быстрее обычного обучения, не подтверждается. Ступенчатый режим с разогревом при том же числе итераций даёт ту же точность (92,26 ± 0,38 %), а по протоколу статьи – 94,02 %, больше, чем CLR. Стандартное расписание из [2] достигает 92,62 % за 164 эпохи.
3) Преимущество CLR при равном бюджете проявляется на малых выборках и растёт с уменьшением данных, до 2,5 пунктов на 10 000 примерах, но оно в несколько раз меньше заявленного в статье.
4) Регуляризующее действие большого learning rate видно по росту потерь на обучении в LR range test и по взаимодействию с weight decay, но уменьшения ошибки обобщения, показанного в статье, не наблюдается.
5) Выигрыш, приведённый в статье, в значительной мере объясняется слабым и слишком длинным бейзлайном.

Список литературы

1) Smith L. N., Topin N. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates // arXiv:1708.07120. – 2018.
2) He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). – 2016. – P. 770–778.
3) Smith L. N. Cyclical Learning Rates for Training Neural Networks // Proc. IEEE Winter Conference on Applications of Computer Vision (WACV). – 2017.
4) Goyal P., Dollár P., Girshick R. et al. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour // arXiv:1706.02677. – 2017.
5) van Laarhoven T. L2 Regularization versus Batch and Weight Normalization // arXiv:1706.05350. – 2017.
6) Krizhevsky A. Learning Multiple Layers of Features from Tiny Images: Technical report. – University of Toronto, 2009.
7) Gulcehre C., Sotelo J., Moczulski M., Bengio Y. A Robust Adaptive Stochastic Gradient Method for Deep Learning // arXiv:1703.00788. – 2017.
8) Idelbayev Y. Proper ResNet Implementation for CIFAR10/CIFAR100 in PyTorch. – URL: https://github.com/akamaster/pytorch_resnet_cifar10

Приложение А. Воспроизведение экспериментов

Код, логи и рисунки находятся в репозитории работы:

– `src/hw_2/` – загрузка данных, модели, расписания learning rate, оценка по формуле (1), обучение (`train.py`) и графики (`plots.py`);

– `scripts/stage2.sh` и `scripts/stage2_subsets_pc.sh` – очереди всех запусков;

– `scripts/make_figures.py` – построение рисунков отчёта;

– `results/<запуск>/` – параметры запуска (`config.json`), кривые обучения (`metrics.csv`) и итоговые метрики (`summary.json`).

Окружение устанавливается командой `uv sync`. Примеры запусков:

```
uv run python -m hw_2.train --schedule onecycle \
    --lr 0.1 --max-lr 3 --iters 10000
uv run python -m hw_2.train --schedule piecewise \
    --lr 0.35 --warmup-iters 200 --iters 80000
uv run python -m hw_2.train --schedule range \
    --lr 0 --max-lr 3 --iters 5000 --eval-every 100
bash scripts/stage2.sh
bash scripts/stage2_subsets_pc.sh
uv run python scripts/make_figures.py
```